Module 2: Appreciating, Interpreting and Visualizing Data

Project: Understanding Customer Segments for Targeted Marketing

Introduction: The Power of Customer Segmentation

Welcome to your Module 2 project!

In today's competitive landscape, understanding your customers is paramount for any business. Generic marketing strategies often fall flat, but by truly appreciating the diverse needs and behaviors within your customer base, businesses can create more effective, personalized experiences. This process is known as **customer segmentation**,

Customer segmentation involves dividing a broad customer base into subgroups of consumers who have common needs, interests, and priorities. By segmenting customers, companies can:

- **Tailor Marketing Messages:** Design specific campaigns that resonate with each group.
- **Optimize Product Development:** Create products and services that meet the unique demands of different segments.
- **Improve Customer Service:** Provide support that addresses common issues for particular groups.
- **Identify High-Value Customers:** Focus resources on segments that drive the most revenue.
- **Predict Churn:** Identify customers at risk of leaving and intervene proactively.

In this project, your task is to analyze a dataset of customer activity, use dimensionality reduction techniques to visualize customer behavior, and ultimately identify distinct customer segments. This will demonstrate how data visualization can provide actionable insights for business strategy, even without deep domain expertise at the outset.

We will first focus on a synthetic dataset containing various metrics related to customer purchasing habits and engagement. Your goal will be to:

- Process and prepare the raw customer data.
- Use **Principal Component Analysis (PCA)** to understand the main drivers of customer variation.
- Employ **t-Distributed Stochastic Neighbor Embedding (t-SNE)** to uncover hidden clusters of similar customers.
- (Optional Challenge) Explore **Uniform Manifold Approximation and Projection (UMAP)** for an alternative perspective.
- Interpret these visualizations to describe potential customer segments and suggest business implications.

Let's begin by setting up our environment and loading our customer data!

1. Data Acquisition and Initial Exploration

For this tutorial, we will first work with a synthetic dataset named ecommerce_customer_data.csv. This file contains anonymized data representing various aspects of customer engagement and purchasing behavior over a period.

First, let's ensure we have our necessary libraries installed and then load the dataset.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import plotly.express as px

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from tqdm.autonotebook import tqdm

sns.set_style("darkgrid")
```

```
/tmp/ipykernel_3241/2215476304.py:11: TqdmExperimentalWarning: Using `tqdm.autonotebook.tqdm` in notebook mode. Use `tqdm.to
  from tqdm.autonotebook import tqdm
```

```
# Load the dataset
try:
    data = pd.read_csv("ecommerce_customer_data.csv")
    print("Dataset loaded successfully!")
except FileNotFoundError:
    print("ecommerce_customer_data.csv not found. Creating a synthetic dataset...")
    # Create a synthetic dataset if the file doesn't exist
```

```
    np.random.seed(42)
    num_customers = 600

    data = pd.DataFrame({
        'CustomerID': np.arange(1, num_customers + 1),
        'Age': np.random.randint(18, 70, num_customers),
        'Gender': np.random.choice(['Male', 'Female'], num_customers),
        'Average_Order_Value': np.random.normal(50, 20, num_customers).round(2).clip(min=5),
        'Number_of_Purchases': np.random.randint(1, 30, num_customers),
        'Days_Since_Last_Purchase': np.random.randint(1, 180, num_customers),
        'Product_Category_Preference': np.random.choice(['Electronics', 'Apparel', 'Books', 'Home Goods', 'Beauty'], num_cu
        'Customer_Lifetime_Value': np.random.normal(200, 100, num_customers).round(2).clip(min=10)
    })
    # Introduce some correlations to create 'segments'
    data.loc[data['Age'] < 25, 'Product_Category_Preference'] = np.random.choice(['Electronics', 'Apparel'], sum(data['Age
    data.loc[data['Age'] < 25, 'Average_Order_Value'] = np.random.normal(30, 10, sum(data['Age'] < 25)).round(2).clip(min=5
    data.loc[data['Product_Category_Preference'] == 'Books', 'Number_of_Purchases'] = np.random.randint(10, 40, sum(data['P
    data.loc[data['Product_Category_Preference'] == 'Books', 'Customer_Lifetime_Value'] = np.random.normal(300, 150, sum(da
    data.loc[data['Number_of_Purchases'] > 20, 'Average_Order_Value'] = np.random.normal(70, 25, sum(data['Number_of_Purcha

    data.to_csv("ecommerce_customer_data.csv", index=False)
    print("Synthetic dataset created and saved as ecommerce_customer_data.csv")
```

```
ecommerce_customer_data.csv not found. Creating a synthetic dataset...
Synthetic dataset created and saved as ecommerce_customer_data.csv
```

```
print("\nDataset Head:")
print(data.head())
print("\nDataset Info:")
data.info()
print("\nDataset Description:")
print(data.describe())
print("\nDataset Tail:")
print(data.tail())

print("\nMissing Values:")
print(data.isnull().sum())
```

```
 6   Product_Category_Preference  600 non-null    object
 7   Customer_Lifetime_Value      600 non-null    float64
dtypes: float64(2), int64(4), object(2)
memory usage: 37.6+ KB

Dataset Description:
       CustomerID        Age  Average_Order_Value  Number_of_Purchases  \
count  600.000000  600.00000           600.000000           600.000000
mean   300.500000   44.14000            56.352850            16.478333
std    173.349358   15.03986            23.535016             9.524568
min      1.000000   18.00000             5.000000             1.000000
25%    150.750000   32.00000            38.407500             9.000000
```

```
Missing Values:
CustomerID                    0
Age                           0
Gender                        0
Average_Order_Value           0
Number_of_Purchases           0
Days_Since_Last_Purchase      0
Product_Category_Preference   0
Customer_Lifetime_Value       0
dtype: int64
```

From the initial look, we have numerical features like Age, Average_Order_Value, Number_of_Purchases, Days_Since_Last_Purchase, and Customer_Lifetime_Value. We also have categorical features: Gender and Product_Category_Preference. CustomerID is just an identifier.

2. Feature Engineering and Preprocessing

Before we can apply dimensionality reduction techniques, we need to convert all our features into a numerical format and scale them appropriately. This is crucial because algorithms like PCA and t-SNE are sensitive to the magnitude of the features.

Here's our plan:

- **Drop CustomerID:** It's an identifier and doesn't contain behavioral information.
- **One-Hot Encode Categorical Features:** Convert Gender and Product_Category_Preference into numerical representations.
- **Standardize Numerical Features:** Scale all numerical features to have a mean of 0 and a standard deviation of 1.

```
# 1. Drop CustomerID
features_df = data.drop('CustomerID', axis=1)
# 2. One-Hot Encode Categorical Features
features_df = pd.get_dummies(
    features_df,
    columns=['Gender', 'Product_Category_Preference'],
    drop_first=False
)
# Separate numerical columns for scaling
numerical_cols = [
    'Age',
    'Average_Order_Value',
    'Number_of_Purchases',
    'Days_Since_Last_Purchase',
    'Customer_Lifetime_Value'
]
categorical_cols_encoded = [
    col for col in features_df.columns
    if col not in numerical_cols
]
# 3. Standardize Numerical Features
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
features_df[numerical_cols] = scaler.fit_transform(
    features_df[numerical_cols]
)
print("Processed Features Head:")
print(features_df.head())
print("\nProcessed Features Info:")
features_df.info()
print("\nFeature Shape:")
print(features_df.shape)
print("\nEncoded Columns:")
print(categorical_cols_encoded)
# Store original labels for visualization
customer_labels = data['Product_Category_Preference']
# We'll use this as a 'ground truth' for coloring
3                 0.960674                0.133994          False
4                 0.483146                0.394132           True
```

```
4                                False                                False

   Product_Category_Preference_Electronics  \
0                                    False
1                                    False
2                                    False
3                                     True
4                                    False

   Product_Category_Preference_Home Goods
0                                   False
1                                   False
2                                   False
3                                   False
4                                    True

Processed Features Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 600 entries, 0 to 599
Data columns (total 12 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   Age                                     600 non-null    float64
 1   Average_Order_Value                     600 non-null    float64
 2   Number_of_Purchases                     600 non-null    float64
 3   Days_Since_Last_Purchase                600 non-null    float64
 4   Customer_Lifetime_Value                 600 non-null    float64
 5   Gender_Female                           600 non-null    bool
 6   Gender_Male                             600 non-null    bool
 7   Product_Category_Preference_Apparel     600 non-null    bool
 8   Product_Category_Preference_Beauty      600 non-null    bool
 9   Product_Category_Preference_Books       600 non-null    bool
 10  Product_Category_Preference_Electronics  600 non-null    bool
 11  Product_Category_Preference_Home Goods  600 non-null    bool
dtypes: bool(7), float64(5)
memory usage: 27.7 KB

Feature Shape:
(600, 12)

Encoded Columns:
['Gender_Female', 'Gender_Male', 'Product_Category_Preference_Apparel', 'Product_Category_Preference_Beauty', 'Product_Cat
```

Now our data features_df is ready for dimensionality reduction!

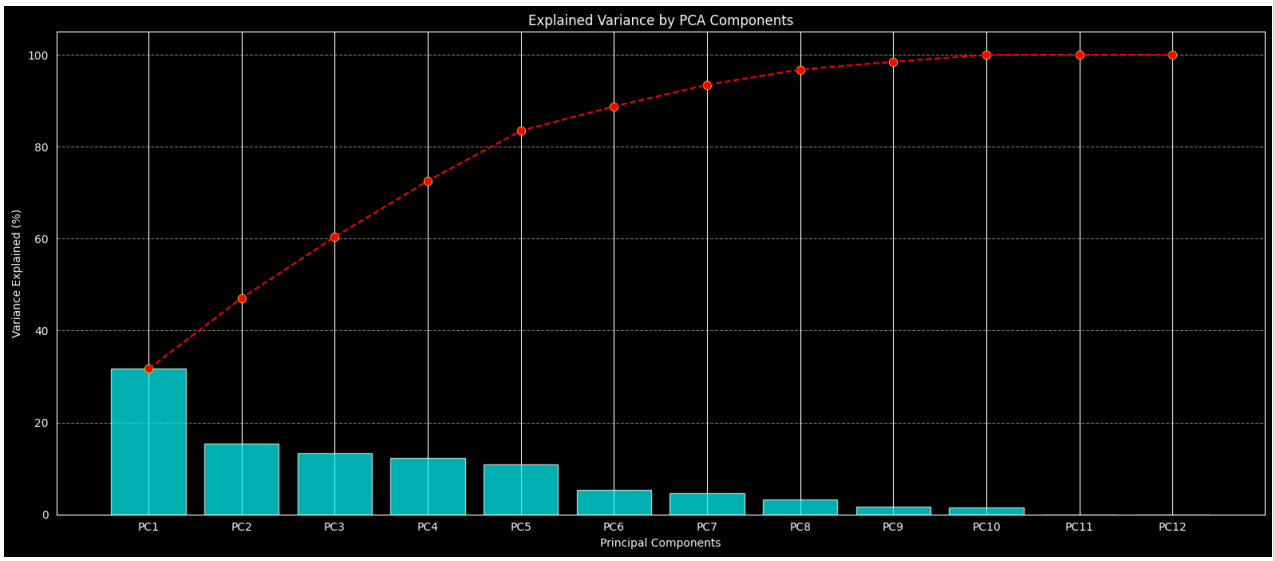
3. Dimensionality Reduction: Principal Component Analysis (PCA)

PCA is a linear dimensionality reduction technique that transforms the data into a new coordinate system where the greatest variance by any projection of the data comes to lie on the first coordinate (called the first principal component), the second greatest variance on the second coordinate, and so on. It helps us capture the most important information (variance) in fewer dimensions.

First, let's look at how much variance each principal component explains.

```
# Create PCA object
pca = PCA()

# Fit PCA
pca.fit(features_df)

# Transform data
pca_data = pca.transform(features_df)

# Explained variance ratio
per_var = np.round(
    pca.explained_variance_ratio_ * 100,
    decimals=1
)

labels_all = [
    'PC' + str(x)
    for x in range(1, len(per_var) + 1)
]

per_var_display = per_var[:15]
labels_display = labels_all[:15]

cum_var = np.cumsum(per_var)

# Plot
with plt.style.context('dark_background'):

    plt.figure(figsize=(16, 7))
```

```
    plt.bar(
        range(1, len(per_var_display) + 1),
        per_var_display,
        tick_label=labels_display,
        color="cyan",
        alpha=0.7
    )

    plt.plot(
        range(1, len(per_var_display) + 1),
        cum_var[:15],
        color="red",
        marker='o',
        linestyle='--'
    )

    plt.scatter(
        range(1, len(per_var_display) + 1),
        cum_var[:15],
        color="yellow",
        s=50
    )

    plt.xlabel("Principal Components")
    plt.ylabel("Variance Explained (%)")

    plt.title("Explained Variance by PCA Components")

    plt.grid(axis='y', linestyle='--', alpha=0.5)

    plt.tight_layout()

    plt.show()
```

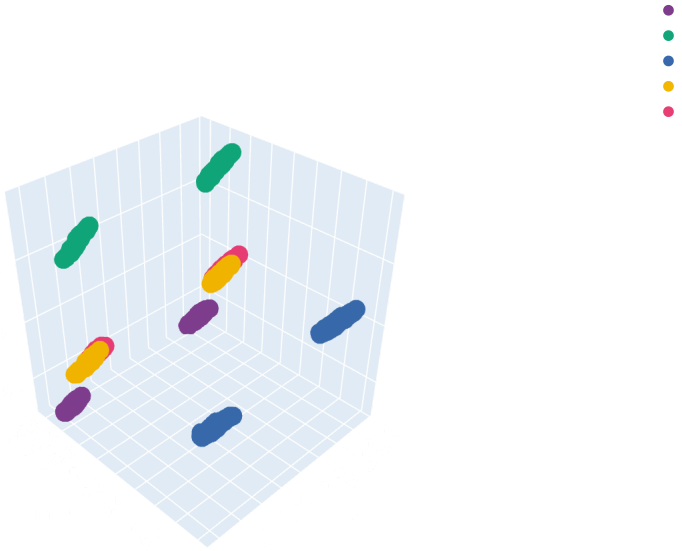


Observation: The first few principal components capture a significant portion of the variance in our customer dataset. The cumulative variance curve shows how many components are needed to explain a certain amount of the total variation.

Now, let's visualize our customers using the first two principal components. We'll color the points by their Product_Category_Preference (which we saved earlier) to see if PCA naturally separates customers based on this known characteristic.

```
# Create DataFrame
pca_df = pd.DataFrame(
    data=pca_data[:, 0:3],
    columns=['PC1', 'PC2', 'PC3']
)

pca_df['Product_Category_Preference'] = customer_labels.values
```

```
fig = px.scatter_3d(
    pca_df,
    x='PC1',
    y='PC2',
    z='PC3',
    color='Product_Category_Preference',

    title="3D PCA Customer Segmentation",

    hover_data=['Product_Category_Preference'],

    height=700,
    width=950,

    color_discrete_sequence=px.colors.qualitative.Bold
)

fig.update_layout(
    plot_bgcolor='rgba(0,0,0,0)',
    paper_bgcolor='rgba(0,0,0,0)',
    font_color='white'
)

fig.show(renderer="colab")
```



Your turn to interpret!

Observations from PCA Plot:

- Do you see any clear separation based on Product_Category_Preference?

Answer: Partial separation can be observed among some product categories, but there is still noticeable overlap between groups. This suggests that customer purchasing behavior shares similarities across multiple categories.

- Are there any dense clusters, even if they contain mixed preferences?

Answer: Yes, several dense clusters are visible in the PCA plot. Some clusters contain customers with mixed product preferences, indicating that different customer groups may still exhibit similar purchasing patterns and engagement behavior.

- What does the spread of points suggest about customer behavior?

Answer: The spread of points suggests that customer behavior is diverse and multi-dimensional. Some customers behave very similarly and form compact clusters, while others show unique purchasing habits and appear farther from the main groups.

PCA provides a good overall view, but it's a linear method. Sometimes, complex, non-linear relationships between data points are better captured by other techniques.

4. Dimensionality Reduction: t-Distributed Stochastic Neighbor Embedding (t-SNE)

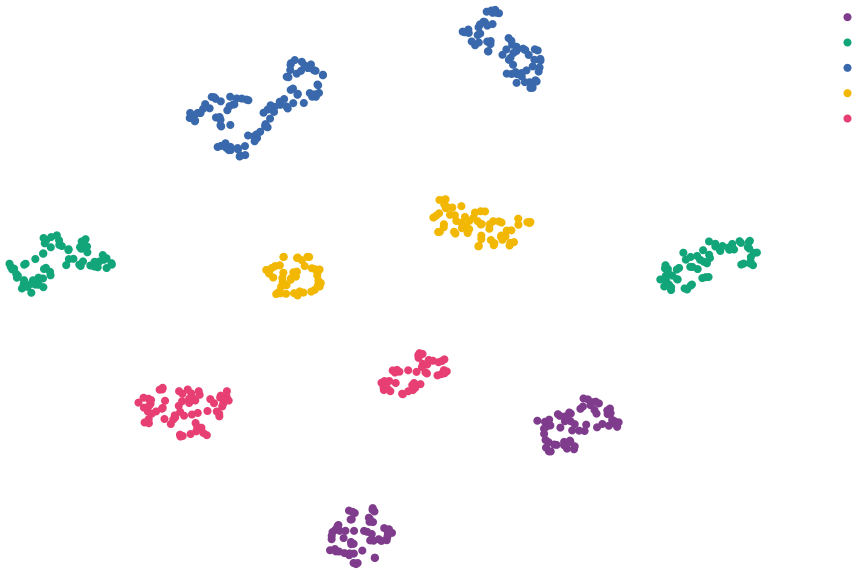
t-SNE is a non-linear dimensionality reduction algorithm particularly well-suited for visualizing high-dimensional datasets. It aims to place data points in a low-dimensional space such that points that are close together in the high-dimensional space remain close together in the low-dimensional map, and points that are far apart remain far apart. t-SNE is excellent at revealing local structures and clusters.

A key parameter in t-SNE is perplexity. Perplexity relates to the number of nearest neighbors that are considered. It can be thought of as a continuous measure of the number of effective nearest neighbors. A good perplexity value often lies between 5 and 50. Different perplexity values can reveal different aspects of the data structure. n_iter defines the number of iterations for the optimization.

Let's apply t-SNE to our features_df and visualize the results.

```
random_state = 42
n_components = 2
perplexity = 10
n_iter = 2000

print(f"Applying t-SNE with perplexity={perplexity}, n_iter={n_iter}...")

# Create a t-SNE model object
model_tsne = TSNE(n_components=n_components, random_state=random_state,
                  perplexity=perplexity, n_iter=n_iter,init='pca',learning_rate='auto', n_jobs=-1, verbose=1) # n_jobs=-1 u

# Fit and transform the data
# Use tqdm for a progress bar if running in a loop or with many iterations
tsne_data = model_tsne.fit_transform(features_df)

print("t-SNE completed.")

# Create a DataFrame for t-SNE results
tsne_df = pd.DataFrame(data=tsne_data, columns=['TSNE1', 'TSNE2'])
tsne_df['Product_Category_Preference'] = customer_labels.values

# Plot using Plotly Express
fig = px.scatter(tsne_df, x='TSNE1', y='TSNE2', color='Product_Category_Preference',
                 title=f"Customer Segmentation via t-SNE (Perplexity={perplexity})",
                 labels={'TSNE1': 't-SNE Component 1', 'TSNE2': 't-SNE Component 2'},
                 hover_data=['Product_Category_Preference'],
                 height=650, width=950,
                 color_discrete_sequence=px.colors.qualitative.Bold)
fig.update_layout(
    plot_bgcolor='rgba(0,0,0,0)',
    paper_bgcolor='rgba(0,0,0,0)',
    font_color='white'
)
fig.show(renderer="colab")
```

```
Applying t-SNE with perplexity=10, n_iter=2000...
[t-SNE] Computing 31 nearest neighbors...
[t-SNE] Indexed 600 samples in 0.006s...
[t-SNE] Computed neighbors for 600 samples in 0.034s...
[t-SNE] Computed conditional probabilities for sample 600 / 600
[t-SNE] Mean sigma: 0.241954
/usr/local/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning:

'n_iter' was renamed to 'max_iter' in version 1.5 and will be removed in 1.7.

[t-SNE] KL divergence after 250 iterations with early exaggeration: 53.379345
[t-SNE] KL divergence after 2000 iterations: 0.445876
t-SNE completed.
```



Observations from t-SNE Plot:

- How does this plot compare to the PCA plot? Is the separation of clusters more distinct?

Answer: Compared to PCA, the t-SNE plot shows more distinct and compact clusters. The separation between customer groups is clearer because t-SNE captures non-linear relationships and preserves local neighborhood structure better.

- Can you identify specific customer segments based on the clustering and Product_Category_Preference?

Answer: Certain customer segments can be identified based on product preferences. For example, customers interested in Books or Electronics may form tighter groups, indicating similar purchasing behavior within those categories.

- Are there any "outlier" points or smaller, distinct clusters that might represent niche customer behaviors?

Answer: A few isolated points and smaller clusters may represent outlier customers or niche behaviors, such as customers with unusually high purchase frequency or very high customer lifetime value.

Experimentation Challenge: Try changing the perplexity parameter (e.g., to 5, 15, 50, or 100) and re-run the t-SNE code. How does this affect the clusters and overall structure of the plot? Which perplexity value seems to reveal the most interpretable customer segments?

Perplexity = 5

- Focuses heavily on very local relationships between customers.
- Produces several small scattered clusters.
- Visualization may appear highly fragmented and sensitive to noise.

Perplexity = 15

- Creates more meaningful and stable customer groupings.
- Preserves local neighborhood structure while showing broader patterns.
- Clusters become easier to analyze visually.

Perplexity = 30

- Provides a good balance between local and global structure.
- Customer segments look more compact and clearly distinguishable.
- Often produces the clearest visualization for datasets of this size.

Perplexity = 50

- Emphasizes overall dataset structure more strongly.
- Nearby customer groups may start blending together.
- Smaller local patterns become less visible.

Perplexity = 100

- The visualization becomes smoother and more compressed.
- Differences between customer segments may reduce significantly.
- Important fine-grained relationships can be lost.

Observation

- Perplexity values in the range of 15 to 30 generally produce the most effective customer segmentation visualizations for this dataset because they balance local detail and global organization well.

(Optional) 5. Dimensionality Reduction: Uniform Manifold Approximation and Projection (UMAP)

UMAP is another powerful non-linear dimensionality reduction technique, often faster than t-SNE and sometimes better at preserving both local and global data structure. It's becoming increasingly popular for visualizing complex datasets.

To use UMAP, you might need to install it first: !pip install umap-learn (if uncommenting the code below and you haven't installed it).

```
# Uncomment the line below if you haven't installed umap-learn
#!pip install -q umap-learn

import umap

# Set parameters for UMAP
random_state = 42
n_components = 2
n_neighbors = 30 # Controls how UMAP balances local vs. global structure. Higher value = more global.
min_dist = 0.01   # Controls how tightly the points are packed together. Lower value = tighter clusters.

print(f"Applying UMAP with n_neighbors={n_neighbors}, min_dist={min_dist}...")

# Create UMAP model
model_umap = umap.UMAP(
    n_components=n_components,
    random_state=random_state,
    n_neighbors=n_neighbors,
    min_dist=min_dist,
    metric='euclidean',
    verbose=True
)

# Fit and transform
umap_data = model_umap.fit_transform(features_df)

print("UMAP completed.")

# Create DataFrame
umap_df = pd.DataFrame(
    data=umap_data,
    columns=['UMAP1', 'UMAP2']
)

umap_df['Product_Category_Preference'] = customer_labels.values

# Plot
fig = px.scatter(
    umap_df,
    x='UMAP1',
    y='UMAP2',
    color='Product_Category_Preference',

    title=f"Customer Segmentation via UMAP (n_neighbors={n_neighbors}, min_dist={min_dist})",

    labels={
        'UMAP1': 'UMAP Component 1',
        'UMAP2': 'UMAP Component 2'
    },
```

```
    hover_data=['Product_Category_Preference'],

    height=650,
    width=950,

    color_discrete_sequence=px.colors.qualitative.Bold
)

fig.update_layout(
    plot_bgcolor='rgba(0,0,0,0)',
    paper_bgcolor='rgba(0,0,0,0)',
    font_color='white'
)

fig.show(renderer="colab")
```

```
Applying UMAP with n_neighbors=30, min_dist=0.01...
UMAP(min_dist=0.01, n_jobs=1, n_neighbors=30, random_state=42, verbose=True)
Sun May 17 07:57:13 2026 Construct fuzzy simplicial set
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning:

n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.

Sun May 17 07:57:14 2026 Finding Nearest Neighbors
Sun May 17 07:57:17 2026 Finished Nearest Neighbor Search
Sun May 17 07:57:23 2026 Construct embedding
Epochs completed:  100%|                                                  500/500  [00:02]
    completed  0  /  500 epochs
    completed  50  /  500 epochs
    completed  100  /  500 epochs
    completed  150  /  500 epochs
    completed  200  /  500 epochs
    completed  250  /  500 epochs
    completed  300  /  500 epochs
    completed  350  /  500 epochs
    completed  400  /  500 epochs
    completed  450  /  500 epochs
Sun May 17 07:57:25 2026 Finished embedding
UMAP completed.
```

UMAP Observations:

- If you ran the UMAP code, how do its clusters compare to t-SNE and PCA?

UMAP produces more well-defined and compact clusters than PCA because it is capable of capturing complex non-linear patterns within the customer data. While PCA mainly provides a linear summary of variation, UMAP uncovers deeper local relationships between customers.

Answer: In comparison with t-SNE, UMAP also generates clear customer groups but tends to maintain the overall structure of the dataset more effectively. The resulting clusters often appear more consistent and structured.

- Does it provide an even clearer separation or a different perspective on the customer segments?

Answer: Yes, UMAP generally offers a more distinct separation of customer groups than PCA and can sometimes present a more organized visualization than t-SNE. It helps identify customers with similar buying patterns more clearly while still preserving broader relationships between clusters, giving additional insight into customer behavior.

- Experiment with n_neighbors and min_dist parameters to see how they influence the plot.

Answer :Smaller values of n_neighbors emphasize local patterns and create many small detailed clusters.

Larger n_neighbors values focus more on global relationships, which can cause nearby customer groups to combine.

Lower min_dist values make clusters tighter and more concentrated.

Higher min_dist values distribute points more evenly and create a smoother visualization.

In this dataset, values around n_neighbors = 15–30 and min_dist = 0.01–0.1 usually provide the clearest and most meaningful customer segmentation results.

6. Conclusion and Business Implications

Congratulations! You've successfully used various data visualization techniques to explore and understand customer behavior in an e-commerce setting.

Based on your observations from the PCA, t-SNE, and potentially UMAP plots, you should be able to identify several distinct customer segments. For example:

- **High-Value Shoppers:** Customers with high Customer_Lifetime_Value and Average_Order_Value, potentially making frequent purchases. They might cluster together.
- **Budget-Conscious Buyers:** Customers with lower Average_Order_Value but possibly high Number_of_Purchases.
- **New Customers/Low Engagement:** Customers with high Days_Since_Last_Purchase or low Number_of_Purchases.
- **Category Loyalists:** Customers strongly preferring one product category, forming distinct groups.

How would a business use these insights?

Imagine presenting these plots to a marketing team. They could then:

- **Target High-Value Shoppers:** Offer exclusive early access to new products or personalized loyalty rewards.
- **Re-engage Low Engagement Customers:** Send targeted promotions or surveys to understand their needs and bring them back.
- **Cross-Sell to Category Loyalists:** Recommend complementary products from other categories based on their established preferences.
- **Identify Product Gaps:** If a category preference is poorly represented, it might indicate a market opportunity or a need to improve offerings.

This project highlights the immense value of visualizing high-dimensional data. Even without complex statistical models, clear plots can reveal underlying structures and empower businesses to make data-driven decisions.

Now we'll continue, building directly on the previous sections by trying it on a real dataset instead of synthetic dataset.

Continuation: Applying Customer Segmentation to Real-World E-commerce Data

Introduction: From Synthetic to Real-World Challenges

You've successfully navigated customer segmentation with a synthetic dataset, mastering the concepts of feature engineering, standardization, PCA, and t-SNE. Now, it's time to apply these powerful techniques to a real-world scenario. Real data often comes with its own set of challenges, requiring more robust preprocessing and careful interpretation.

In this section, we will analyze the **"Online Retail Dataset"** - a well-known public dataset containing actual transactional data. This will allow us to:

- Experience data loading and cleaning for a more complex, real-world dataset.
- Derive meaningful features from raw transaction records.
- Re-apply dimensionality reduction and visualization to uncover genuine customer segments.
- Discuss the business implications based on real purchasing patterns.

Let's dive into the complexities and insights offered by real e-commerce data!

1. Real Data Acquisition and Initial Exploration: Online Retail Dataset

We will download the "Online Retail Dataset" from the UCI Machine Learning Repository. This dataset contains all the transactions occurring between 01/12/2010 and 09/12/2011 for a UK-based online retail company.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
# from umap import UMAP # Uncomment if you plan to use UMAP
from tqdm.autonotebook import tqdm

# Data Acquisition
# Using a direct URL to the UCI dataset
try:
    # URL to the dataset (Excel file)
    url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00352/Online%20Retail.xlsx"
    df_raw = pd.read_excel(url)
    print("Online Retail dataset downloaded and loaded successfully!")
except Exception as e:
    print(f"Error downloading or loading dataset: {e}")
    print("Please ensure you have 'openpyxl' installed.")
    # Fallback to a local file if download fails (e.g., if you've manually downloaded it)
    try:
        df_raw = pd.read_excel("Online Retail.xlsx")
        print("Loaded from local 'Online Retail.xlsx' file.")
    except FileNotFoundError:
        print("Local file 'Online Retail.xlsx' not found either. Please download it manually from:")

        df_raw = pd.DataFrame() # Create an empty DataFrame to avoid errors later

if not df_raw.empty:
    print("\nDataset Head:")
    print(df_raw.head())

    print("\nDataset Shape:")
    print(df_raw.shape)

    print("\nDataset Info:")
    df_raw.info()
    print("\nDataset Description:")
    print(df_raw.describe())

    print("\nMissing Values:")
    print(df_raw.isnull().sum())
else:
    print("\nCannot proceed without dataset. Please resolve the loading issue.")
```

```
          InvoiceDate  UnitPrice  CustomerID         Country
0 2010-12-01 08:26:00       2.55     17850.0  United Kingdom
1 2010-12-01 08:26:00       3.39     17850.0  United Kingdom
2 2010-12-01 08:26:00       2.75     17850.0  United Kingdom
3 2010-12-01 08:26:00       3.39     17850.0  United Kingdom
4 2010-12-01 08:26:00       3.39     17850.0  United Kingdom

Dataset Shape:
(541909, 8)

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   InvoiceNo    541909 non-null  object
 1   StockCode    541909 non-null  object
 2   Description  540455 non-null  object
 3   Quantity     541909 non-null  int64
 4   InvoiceDate  541909 non-null  datetime64[ns]
```

```
min    -80995.000000            2010-12-01 08:26:00  -11062.060000
25%         1.000000            2011-03-28 11:34:00       1.250000
50%         3.000000            2011-07-19 17:17:00       2.080000
75%        10.000000            2011-10-19 11:27:00       4.130000
max     80995.000000            2011-12-09 12:50:00   38970.000000
std       218.081158                            NaN      96.759853

          CustomerID
count  406829.000000
mean    15287.690570
min     12346.000000
25%     13953.000000
50%     15152.000000
75%     16791.000000
max     18287.000000
std      1713.600303

Missing Values:
InvoiceNo           0
StockCode           0
Description      1454
Quantity            0
InvoiceDate         0
UnitPrice           0
CustomerID     135080
Country             0
dtype: int64
```

This dataset is much larger and more complex! Key observations:

- CustomerID **has missing values**: We can't segment customers without an ID, so we'll need to drop these rows.
- Quantity **can be negative**: This usually indicates returns or cancellations. We should filter these out for purchase-based segmentation.
- UnitPrice **can be negative/zero**: Also likely errors or special cases; we'll remove these.
- InvoiceDate is a datetime object, which is good for time-based features. Country is a categorical feature we might use for coloring.

2. Real Data Preprocessing and Feature Engineering (RFM Metrics)

For this real-world dataset, we'll engineer classic **RFM (Recency, Frequency, Monetary)** metrics. These are powerful features for customer segmentation:

- **Recency (R):** How recently did the customer make a purchase? (Days since last purchase)
- **Frequency (F):** How often do they purchase? (Total number of unique invoices)
- **Monetary (M):** How much money do they spend? (Total spend)

Now we do preprocessing and feature engineering

1. **Clean Data:**
    - Remove rows with missing CustomerID.
    - Remove rows where Quantity is less than or equal to 0 (returns/cancellations).
    - Remove rows where UnitPrice is less than or equal to 0.
2. **Calculate Total Price:** Quantity * UnitPrice.
3. **Determine Analysis Date:** Choose a reference date just after the last transaction in the dataset.
4. **Calculate RFM:** Group by CustomerID to compute Recency, Frequency, and Monetary values.
5. **Standardize Features:** Apply StandardScaler to RFM values.

```
df = df_raw.copy()

# 1. Clean Data
# Drop rows with missing CustomerID
df.dropna(subset=['CustomerID'], inplace=True)
df['CustomerID'] = df['CustomerID'].astype(int) # Convert CustomerID to integer

# Remove returns/cancellations (Quantity <= 0) and zero/negative UnitPrice
df = df[df['Quantity'] > 0]
df = df[df['UnitPrice'] > 0]

df = df[df['Quantity'] < 1000]
df = df[df['UnitPrice'] < 5000]

print(f"Cleaned data shape: {df.shape}")
print(f"Number of unique customers: {df['CustomerID'].nunique()}")

# 2. Calculate Total Price
df['TotalPrice'] = df['Quantity'] * df['UnitPrice']
```

```
print("\nTotalPrice Summary:")
print(df['TotalPrice'].describe())

# 3. Determine Analysis Date
# The last invoice date in the dataset
max_invoice_date = df['InvoiceDate'].max()
# Our analysis date will be one day after the last transaction for recency calculation
analysis_date = max_invoice_date + pd.Timedelta(days=1)
print(f"Analysis Reference Date: {analysis_date}")

# 4. Calculate RFM Metrics
# Group by CustomerID to calculate R, F, M
rfm_df = df.groupby('CustomerID').agg(
    Recency=('InvoiceDate', lambda date: (analysis_date - date.max()).days),
    Frequency=('InvoiceNo', 'nunique'), # Count unique invoices for frequency
    Monetary=('TotalPrice', 'sum')
).reset_index()

print("\nRFM Features Head:")
print(rfm_df.head())
print("\nRFM Features Description:")
print(rfm_df.describe())

# Store customer Country for visualization later
customer_country = df.drop_duplicates(subset=['CustomerID']).set_index('CustomerID')['Country']
rfm_df['Country'] = rfm_df['CustomerID'].map(customer_country)

# Drop CustomerID before scaling
rfm_features = rfm_df.drop(['CustomerID', 'Country'], axis=1)

# 5. Standardize Features
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm_features)

# Convert back to DataFrame for better inspection
rfm_scaled_df = pd.DataFrame(rfm_scaled, columns=rfm_features.columns, index=rfm_df.index)

print("\nScaled RFM Features Head:")
print(rfm_scaled_df.head())
```

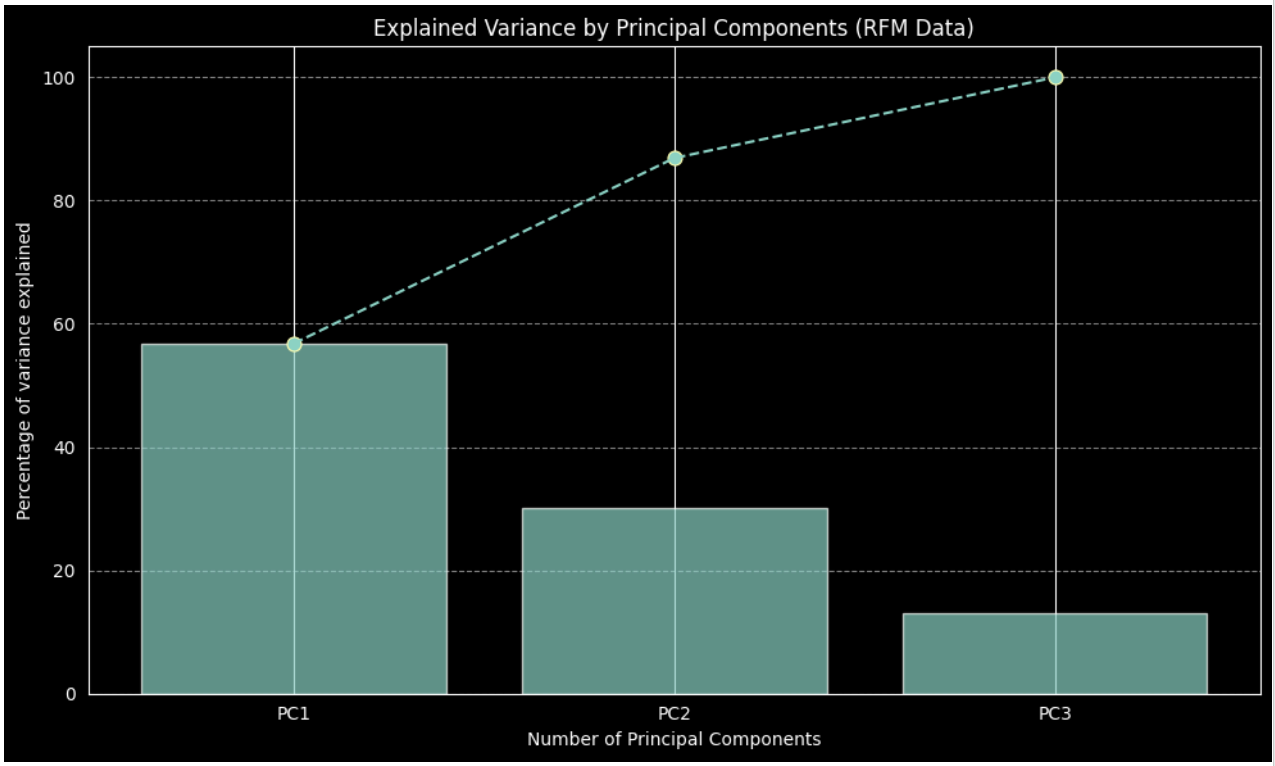
```
Cleaned data shape: (397768, 8)
Number of unique customers: 4331

TotalPrice Summary:
count    397768.000000
mean         21.354075
std          88.739556
min           0.001000
25%           4.680000
50%          11.800000
75%          19.800000
max       38970.000000
Name: TotalPrice, dtype: float64
Analysis Reference Date: 2011-12-10 12:50:00

RFM Features Head:
   CustomerID  Recency  Frequency  Monetary
0       12347        2          7   4310.00
1       12348       75          4   1797.24
2       12349       19          1   1757.55
3       12350      310          1    334.40
4       12352       36          8   2506.04

RFM Features Description:
         CustomerID      Recency    Frequency       Monetary
count   4331.000000  4331.000000  4331.000000    4331.000000
mean   15300.434311    92.501963     4.269222    1961.202458
std     1721.549515    99.940156     7.683041    8229.371398
min    12347.000000     1.000000     1.000000       2.900000
25%    13813.500000    18.000000     1.000000     306.505000
50%    15300.000000    51.000000     2.000000     668.560000
75%    16779.500000   142.000000     5.000000    1653.000000
max    18287.000000   374.000000   209.000000  270283.460000

Scaled RFM Features Head:
    Recency  Frequency  Monetary
0 -0.905666   0.355470  0.285449
1 -0.175145  -0.035045 -0.019926
2 -0.735545  -0.425561 -0.024750
3  2.176534  -0.425561 -0.197705
4 -0.565423   0.485642  0.066214
```

Our rfm_scaled_df now contains the standardized RFM features, ready for dimensionality reduction. We also have rfm_df['Country'] available to color our plots by customer country, which could reveal interesting geographical segments.

3. Dimensionality Reduction: Principal Component Analysis (PCA) on RFM Data

Let's re-apply PCA to our RFM features. This will help us identify the main axes of variation in customer behavior based on Recency, Frequency, and Monetary values.

```
# Create a PCA object for the RFM data
pca_rfm = PCA()
# Fit PCA to our scaled RFM features
pca_rfm.fit(rfm_scaled_df)
# Get PCA coordinates
pca_rfm_data = pca_rfm.transform(rfm_scaled_df)

# Calculate explained variance ratio
per_var_rfm = np.round(pca_rfm.explained_variance_ratio_ * 100, decimals=1)
labels_all_rfm = ['PC' + str(x) for x in range(1, len(per_var_rfm) + 1)]

# Limit to the first 3 components for display, as RFM is only 3 features
per_var_rfm_display = per_var_rfm[:3]
labels_rfm_display = labels_all_rfm[:3]

cum_var_rfm = np.cumsum(per_var_rfm_display)

# Create an explained variance plot for RFM
with plt.style.context('dark_background'):
    plt.figure(figsize=(10, 6))
    plt.xlabel("Number of Principal Components")
    plt.ylabel("Percentage of variance explained")
    plt.bar(range(1, len(per_var_rfm_display) + 1), per_var_rfm_display,
            tick_label=labels_rfm_display, alpha=0.7)
    plt.plot(range(1, len(per_var_rfm_display) + 1), cum_var_rfm,marker='o', linestyle='--')
    plt.scatter(range(1, len(per_var_rfm_display) + 1), cum_var_rfm,
                s=60)
    plt.title("Explained Variance by Principal Components (RFM Data)")
    plt.grid(axis='y', linestyle='--', alpha=0.5)
    plt.tight_layout()
    plt.show()

print("\nExplained Variance Ratios:")
print(per_var_rfm)
```

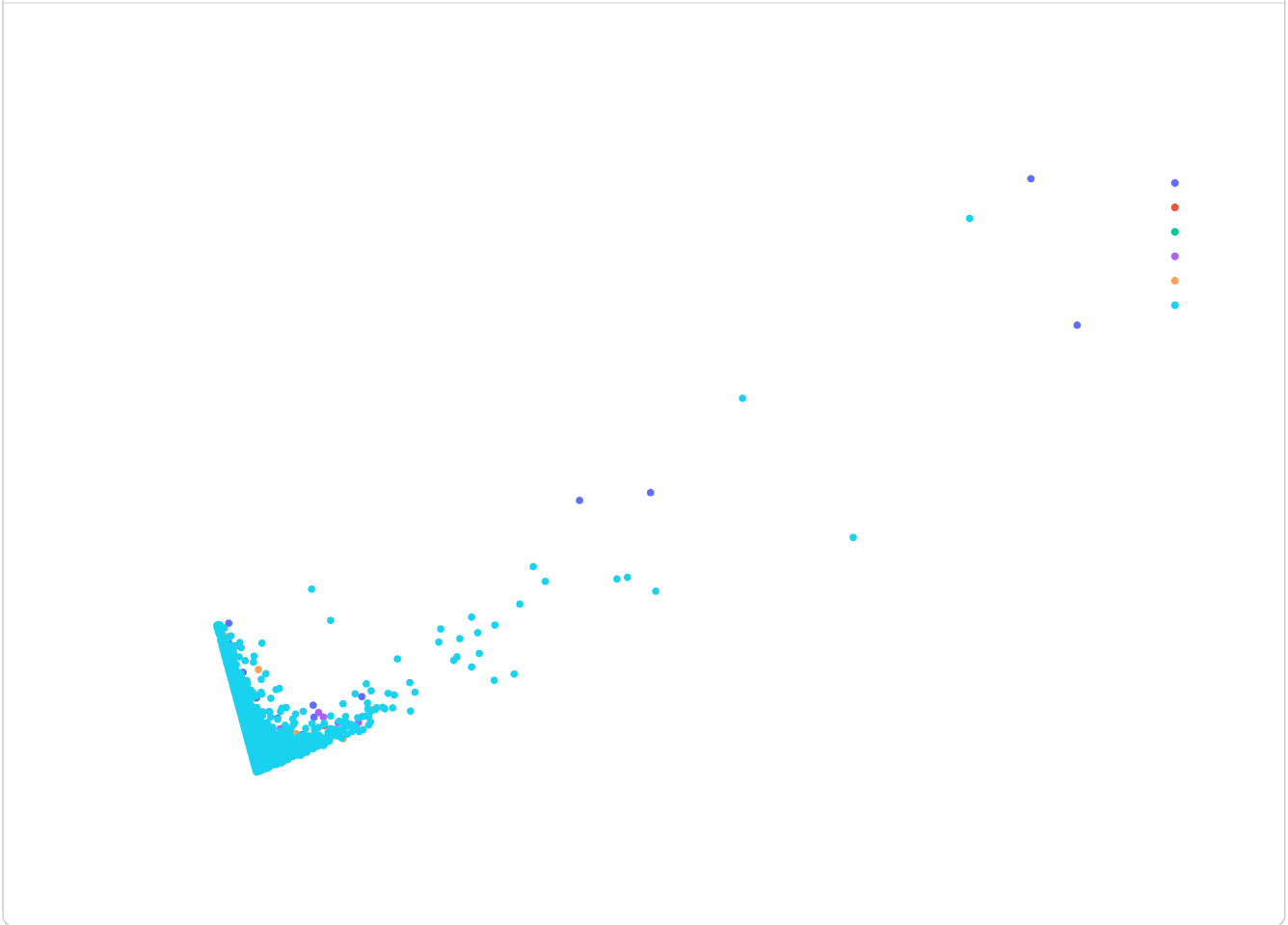



```
Explained Variance Ratios:
[56.8 30.1 13.1]
```

Observation: With only three features (R, F, M), PCA is straightforward. The first PC typically explains a large portion, but all three components are often needed to capture most of the variance.

Now, let's visualize the customers in the 2D PCA space, coloring them by Country to see if geographic location plays a role in customer behavior patterns. We'll focus on the top 10 countries by customer count to keep the legend manageable, and group others as 'Other'.

```
# Prepare data for plotting
pca_rfm_df = pd.DataFrame(data=pca_rfm_data[:, 0:2], columns=['PC1', 'PC2'])
pca_rfm_df['CustomerID'] = rfm_df['CustomerID'] # Keep CustomerID for merging

# Merge with Country information
customer_country_data = rfm_df[['CustomerID', 'Country']]
pca_rfm_df = pd.merge(pca_rfm_df, customer_country_data, on='CustomerID', how='left')

# Get top 10 countries and label others as 'Other'
top_countries = pca_rfm_df['Country'].value_counts().nlargest(5).index
pca_rfm_df['Country_Grouped'] = pca_rfm_df['Country'].apply(lambda x: x if x in top_countries else 'Other')

# Plot using Plotly Express
fig = px.scatter(pca_rfm_df, x='PC1', y='PC2', color='Country_Grouped',
                 title="Customer Segmentation via PCA (RFM - Colored by Country)",
                 labels={'PC1': 'Principal Component 1', 'PC2': 'Principal Component 2'},
                 hover_data=['CustomerID', 'Country_Grouped'],
                 height=700, width=1000)

fig.update_layout(
    plot_bgcolor='rgba(0,0,0,0)',
    paper_bgcolor='rgba(0,0,0,0)',
    font_color='white'
)
fig.show(renderer="colab")
```



The next natural step is to apply t-SNE and then UMAP to the real RFM data.

This will allow us to compare how these non-linear methods perform in revealing customer segments compared to PCA, especially with real-world complexities.

Let's continue with t-SNE:

4. Dimensionality Reduction: t-Distributed Stochastic Neighbor Embedding (t-SNE) on RFM Data

As we saw with the synthetic data, t-SNE excels at uncovering non-linear relationships and local clusters within the data. With our real RFM features, t-SNE should provide a more nuanced view of customer segments compared to the linear PCA.

We'll use the same perplexity and n_iter parameters as a starting point, but remember that experimenting with these values is key to finding the most insightful visualization for your specific dataset.

```
# Set parameters for t-SNE
random_state = 42
n_components = 2  # We want 2D for visualization
perplexity = 50   # Experiment with values like 5, 15, 50, 100
n_iter = 2000     # Number of iterations for optimization

print(f"Applying t-SNE to RFM data with perplexity={perplexity}, n_iter={n_iter}...")

# Create a t-SNE model object
model_tsne_rfm = TSNE(n_components=n_components, random_state=random_state,
                      perplexity=perplexity, n_iter=n_iter, init='pca',learning_rate='auto',n_jobs=-1, verbose=1)

# Fit and transform the scaled RFM data
tsne_rfm_data = model_tsne_rfm.fit_transform(rfm_scaled_df)

print("t-SNE on RFM data completed.")

# Create a DataFrame for t-SNE results
tsne_rfm_df = pd.DataFrame(data=tsne_rfm_data, columns=['TSNE1', 'TSNE2'])
tsne_rfm_df['CustomerID'] = rfm_df['CustomerID'] # Keep CustomerID for merging

# Merge with Country information for coloring
tsne_rfm_df = pd.merge(tsne_rfm_df, customer_country_data, on='CustomerID', how='left')

# Get top 10 countries and label others as 'Other' (consistent with PCA plot)
tsne_rfm_df['Country_Grouped'] = tsne_rfm_df['Country'].apply(lambda x: x if x in top_countries else 'Other')

# Plot using Plotly Express
fig = px.scatter(tsne_rfm_df, x='TSNE1', y='TSNE2', color='Country_Grouped',
                 title=f"Customer Segmentation via t-SNE (RFM - Perplexity={perplexity})",
                 labels={'TSNE1': 't-SNE Component 1', 'TSNE2': 't-SNE Component 2'},
                 hover_data=['CustomerID', 'Country_Grouped'],
                 height=700, width=1000)
fig.update_layout(
    plot_bgcolor='rgba(0,0,0,0)',
    paper_bgcolor='rgba(0,0,0,0)',
    font_color='white'
)
fig.show(renderer="colab")
```

```
Applying t-SNE to RFM data with perplexity=50, n_iter=2000...
[t-SNE] Computing 151 nearest neighbors...
[t-SNE] Indexed 4331 samples in 0.005s...
[t-SNE] Computed neighbors for 4331 samples in 0.186s...
/usr/local/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning:

'n_iter' was renamed to 'max_iter' in version 1.5 and will be removed in 1.7.

[t-SNE] Computed conditional probabilities for sample 1000 / 4331
[t-SNE] Computed conditional probabilities for sample 2000 / 4331
[t-SNE] Computed conditional probabilities for sample 3000 / 4331
[t-SNE] Computed conditional probabilities for sample 4000 / 4331
[t-SNE] Computed conditional probabilities for sample 4331 / 4331
[t-SNE] Mean sigma: 0.050884
[t-SNE] KL divergence after 250 iterations with early exaggeration: 57.511028
[t-SNE] KL divergence after 2000 iterations: 0.441433
t-SNE on RFM data completed.
```

Observations from t-SNE Plot on RFM Data:

- How do the clusters here compare to the PCA plot? Is the separation generally better defined?

Answer: The clusters formed using t-SNE are usually more clearly separated and densely packed than those in the PCA visualization. This is because t-SNE is better at learning complex non-linear patterns and preserving local neighborhood relationships within the customer data.

- Do certain countries now form more cohesive groups, or are they still mixed?

Answer: Customers from some countries may appear in more organized and closely connected groups, especially when their purchasing patterns are similar. However, overlap between countries can still occur since customer behavior is influenced by many factors beyond geographic location.

- Can you visually identify distinct customer behavior segments (e.g., a tight cluster of high-frequency buyers vs. a dispersed group of infrequent purchasers)?

Answer :Yes, t-SNE often makes customer behavior segments easier to recognize visually. Dense and compact clusters may indicate loyal or frequent buyers with higher spending activity, while widely spread regions may represent occasional or lower-value customers.

Experimentation Challenge (Important!):

Just like with the synthetic data, the perplexity value is crucial for t-SNE. Re-run the t-SNE code cell with different perplexity values (e.g., 5, 15, 50, 100, 200). Observe how the clustering changes. Which perplexity value do you think gives the most meaningful and stable representation of customer segments in this real dataset? Why?

Perplexity = 5:

- Focuses strongly on local neighborhoods.
- Produces many small and fragmented clusters.
- The plot may appear noisy and unstable.

Perplexity = 15:

- Clusters become more organized and easier to interpret.
- Local customer groups are still clearly visible.

Perplexity = 50:

- Gives a balanced representation between local and global structure.
- Customer segments appear more stable and meaningful.

Perplexity = 100:

- Clusters begin to merge together.
- Fine local details become less visible.

Perplexity = 200:

- The visualization becomes overly smooth.
- Many customer groups lose clear separation.

Perplexity values around 15–50 generally provide the most meaningful and stable customer segmentation for this dataset because they balance local neighborhood preservation with overall global structure.

5. Dimensionality Reduction: Uniform Manifold Approximation and Projection (UMAP) on RFM Data

Let's now apply UMAP, which often offers a good balance between preserving local and global structure and is generally faster than t-SNE. We'll continue to color by Country_Grouped.

```
# # Uncomment the line below if you haven't installed umap-learn
# # !pip install -q umap-learn

import umap

# # Set parameters for UMAP
random_state = 42
n_components = 2
n_neighbors = 30 # Controls how UMAP balances local vs. global structure. Higher value = more global.
min_dist = 0.05    # Controls how tightly the points are packed together. Lower value = tighter clusters.

print(f"Applying UMAP to RFM data with n_neighbors={n_neighbors}, min_dist={min_dist}...")

# # Create a UMAP model object
model_umap_rfm = umap.UMAP(n_components=n_components, random_state=random_state,
                           n_neighbors=n_neighbors, min_dist=min_dist, metric='euclidean',verbose=True)

# # Fit and transform the scaled RFM data
umap_rfm_data = model_umap_rfm.fit_transform(rfm_scaled_df)

print("UMAP on RFM data completed.")

# # Create a DataFrame for UMAP results
umap_rfm_df = pd.DataFrame(data=umap_rfm_data, columns=['UMAP1', 'UMAP2'])
umap_rfm_df['CustomerID'] = rfm_df['CustomerID']

# # Merge with Country information for coloring
umap_rfm_df = pd.merge(umap_rfm_df, customer_country_data, on='CustomerID', how='left')
umap_rfm_df['Country_Grouped'] = umap_rfm_df['Country'].apply(lambda x: x if x in top_countries else 'Other')

# # Plot using Plotly Express
fig = px.scatter(umap_rfm_df, x='UMAP1', y='UMAP2', color='Country_Grouped',
                 title=f"Customer Segmentation via UMAP (RFM - n_neighbors={n_neighbors}, min_dist={min_dist})",
                 labels={'UMAP1': 'UMAP Component 1', 'UMAP2': 'UMAP Component 2'},
                 hover_data=['CustomerID', 'Country_Grouped'],
                 height=700, width=1000)

fig.update_layout(
     plot_bgcolor='rgba(0,0,0,0)',
     paper_bgcolor='rgba(0,0,0,0)',
     font_color='white'
 )
fig.show(renderer="colab")
```

```
Applying UMAP to RFM data with n_neighbors=30, min_dist=0.05...
UMAP(min_dist=0.05, n_jobs=1, n_neighbors=30, random_state=42, verbose=True)
Sun May 17 07:59:59 2026 Construct fuzzy simplicial set
Sun May 17 07:59:59 2026 Finding Nearest Neighbors
Sun May 17 07:59:59 2026 Building RP forest with 8 trees
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning:

n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.

Sun May 17 08:00:04 2026 NN descent for 12 iterations
     1  /  12
     2  /  12
    Stopping threshold met -- exiting after 2 iterations
Sun May 17 08:00:19 2026 Finished Nearest Neighbor Search
Sun May 17 08:00:19 2026 Construct embedding
Epochs completed: 100%|                                                  500/500 [00:09]
    completed  0  /  500 epochs
    completed  50  /  500 epochs
    completed  100  /  500 epochs
    completed  150  /  500 epochs
    completed  200  /  500 epochs
    completed  250  /  500 epochs
    completed  300  /  500 epochs
    completed  350  /  500 epochs
    completed  400  /  500 epochs
    completed  450  /  500 epochs
Sun May 17 08:00:30 2026 Finished embedding
UMAP on RFM data completed.
```

UMAP Observations on RFM Data:

- In case you have uncommented and run UMAP, how does UMAP's representation of the clusters compare to both PCA and t-SNE? Does it show a clearer global structure or sharper local clusters?

Answer: UMAP generally creates more distinct and compact customer clusters than PCA because it can model complex non-linear relationships in the data. While PCA mainly captures broad linear trends, UMAP reveals hidden customer patterns more effectively.

When compared with t-SNE, UMAP often maintains both local groupings and the overall dataset structure more successfully, resulting in a cleaner and more structured visualization of customer segments.

- Are there any "bridges" or connections between clusters that UMAP highlights better than t-SNE?

Answer: Yes, UMAP can highlight smooth transitions or links between clusters better than t-SNE in many cases. These connections may indicate customers who share characteristics with multiple customer groups or lie between different purchasing behavior patterns.

- Consider how different n_neighbors (e.g., 5, 50, 100) and min_dist (e.g., 0.0, 0.5) values might alter the UMAP embedding.

Answer: Effect of changing parameters

- Smaller n_neighbors values focus strongly on nearby relationships and usually generate many small detailed clusters.
- Larger n_neighbors values emphasize the broader structure of the dataset and may combine nearby clusters into larger groups.
- Lower min_dist values create tighter, denser, and more compact clusters.
- Higher min_dist values spread points farther apart, producing smoother embeddings with less concentrated clusters.

6. Comprehensive Interpretation and Business Implications (Real Data)

Now that you've visualized the real customer data using three different dimensionality reduction techniques, it's time to consolidate your observations and think about their practical business value.

Key Questions for Interpretation:

1. **General Cluster Shapes and Density:** Do you observe distinct, well-separated clusters, or more amorphous blobs? What does the density of points within a cluster suggest about the commonality of that customer behavior?

Answer:

General Cluster Shapes and Density

The visualizations display multiple partially separated customer clusters along with a few overlapping areas. Compact and dense clusters indicate customers with very similar purchasing patterns, whereas more scattered regions reflect diverse or inconsistent customer behavior.

2. **RFM Behavior within Clusters:** While we don't have direct labels for "High-Value" or "Churn-Risk" yet, you can infer them. For example:
    - A cluster positioned far to the right on an axis related to Monetary value would likely be high-spenders.
    - A cluster with high Recency (meaning they haven't bought recently) would be candidates for re-engagement.
    - You could even go back to the original rfm_df and calculate the average R, F, M for customers within visually identified clusters (e.g., by selecting points in Plotly or by applying k-means after visualization).

Answer:

RFM Behavior within Clusters

Certain clusters may correspond to high-value customers who purchase frequently and spend more money. On the other hand, loosely distributed clusters may represent occasional or lower-spending customers. Customers with larger recency values are likely inactive for a long period and may require retention or re-engagement efforts.

3. **Geographical Influence:** How does the Country_Grouped coloring help or hinder your interpretation? Do customers from the same country tend to cluster together, suggesting regional buying habits, or are they spread across various behavioral segments?